# Hostel Management System

Project Documentation & 11-Week Roadmap

| | |
|---|---|
| **Project Title** | Hostel Management System |
| **Tech Stack** | MySQL • Flask • HTML • Bootstrap |
| **Duration** | 11 Weeks |
| **Modules** | 5 Core Modules |
| **Version Control** | Git & GitHub |

## Core Modules

1. Student Management
2. Room Management
3. Room Allocation
4. Fee Management
5. Complaint Management

Department of AI & Data Science
Academic Database Systems Project

# Contents

# 1. Project Overview

**Objective:** Design and develop a web-based Hostel Management System that automates core administrative tasks of a university hostel — including student registration, room allocation, fee collection, and complaint tracking.

## 1.1 Problem Statement

University and college hostels face significant challenges managing student records, room assignments, fee collection, and complaint resolution through manual, paper-based processes. These systems are prone to errors, data loss, and inefficiency. This project solves these problems by providing a centralized, digital hostel management platform accessible through a web browser.

## 1.2 Key Features

- Student registration, search, and profile management
- Room inventory management with real-time capacity tracking
- Automated room-to-student allocation and deallocation
- Fee tracking with payment status and due-date alerts
- Complaint submission, assignment, and resolution workflow
- Dashboard with summary statistics and quick-access panels
- Capacity enforcement via database-level trigger (no overbooking)

## 1.3 Technology Stack

| Layer | Technology |
|---|---|
| **Database** | MySQL 8.x |
| **Backend** | Python 3.x + Flask |
| **Frontend** | HTML5 + Bootstrap 5 |
| **Version Control** | Git + GitHub |

# 2. Module Descriptions

| | |
|---|---|
| **Student Management** | Register, view, update, and remove student records including personal details, contact information, and academic identifiers. |
| **Room Management** | Manage room inventory, configure room types (single/double/dormitory), set capacity limits, and track availability. |
| **Room Allocation** | Assign specific rooms to registered students, record check-in and check-out dates, and manage reallocation. |
| **Fee Management** | Record fee payments, track outstanding dues, categorize fees by type, and generate payment history per student. |
| **Complaint Management** | Allow students to submit complaints, enable staff to assign and resolve issues, and maintain a resolution audit trail. |

# 3. Database Design

## 3.1 Entity-Relationship Summary

The database consists of **5 entities** with clearly defined primary keys and foreign key relationships:

| Entity | Primary Key | Foreign Key(s) | Key Attributes |
|---|---|---|---|
| Students | student_id | — | name, email, phone, course, year |
| Rooms | room_id | — | room_number, type, capacity, status |
| Allocations | allocation_id | student_id, room_id | alloc_date, vacate_date, status |
| Fees | fee_id | student_id | amount, due_date, paid_date, status |
| Complaints | complaint_id | student_id | description, category, status, date |

### 3.2 Relationship Diagram (Text Representation)

```
Student (1)                                          (M) Allocation (M)
                                                 (1) Room
Student (1)                                          (M) Fee
Student (1)                                          (M) Complaint
```

Listing 1: Entity Relationships

## 4. 11-Week Project Roadmap

**Week 1 — Requirement Analysis**

**Tasks:**

- Write the project objective and problem statement
- List all functional requirements per module
- Document the expected system behavior

**GitHub Push:**

```
README.md
Requirement_Analysis.pdf
```

**Week 2 — ER Diagram**

**Tasks:**

- Design ER diagram with 5 entities: Student, Room, Allocation, Fee, Complaint
- Define cardinality (one-to-many relationships)
- Identify all attributes for each entity

**GitHub Push:**

```
ER_Diagram.png
```

## Week 3 — Database Schema

**Tasks:**

- Create all 5 normalized tables in MySQL
- Apply Primary Keys, Foreign Keys, and appropriate constraints
- Use correct data types (`INT`, `VARCHAR`, `DATE`, `DECIMAL`)

**GitHub Push:**

```
schema.sql
```

## Week 4 — Sample Data

**Tasks:**

- Insert 20 realistic student records
- Insert 10 room records with varying types and capacities
- Insert 10 fee records and 10 complaint records

**GitHub Push:**

```
sample_data.sql
```

## Week 5 — Basic SQL Queries

**Tasks:**

- Write queries using: `SELECT`, `WHERE`, `ORDER BY`
- Apply aggregate functions: `COUNT`, `SUM`, `AVG`
- Filter and sort data from single tables

**GitHub Push:**

```
basic_queries.sql
```

## Week 6 — JOIN Queries

**Tasks:**

- Write `INNER JOIN` queries combining Student + Room
- Write `LEFT JOIN` queries combining Student + Fee
- Explore multi-table joins across 3 entities

**GitHub Push:**

```
join_queries.sql
```

## Week 7 — Views

**Tasks:**

- Create `Student_Room_View` combining student and room data
- Optionally create a `Fee_Summary_View`
- Use views in subsequent queries as virtual tables

**GitHub Push:**

```
views.sql
```

## Week 8 — Trigger

**Tasks:**

- Implement **Room Capacity Check** trigger on the Allocations table
- Trigger fires `BEFORE INSERT` and counts current occupancy
- Raises an error signal if room is already at full capacity

**GitHub Push:**

```
trigger.sql
```

## Week 9 — Flask Backend

**Tasks:**

- Set up Flask project structure with blueprints
- Build 4–5 core pages: Dashboard, Students, Rooms, Complaints
- Connect to MySQL using `mysql-connector-python`

**GitHub Push:**

```
backend/
```

## Week 10 — Frontend Integration & CRUD

**Tasks:**

- Build HTML templates with Bootstrap 5
- Implement Add Student, Add Room, Add Complaint forms
- Connect forms to Flask routes and MySQL (full CRUD cycle)

**GitHub Push:**

```
frontend/
```

**Week 11 — Final Testing & Report**

**Tasks:**

- Perform end-to-end testing of all features
- Capture screenshots of all pages and features
- Compile Final Report with ER Diagram, SQL Queries, and explanation
- Prepare presentation slides

**GitHub Push:**

```
Final_Report.pdf
Presentation.pptx
```

# 5. Repository Structure

```
hostel-management-system/
|
|-- README.md
|-- Requirement_Analysis.pdf
|-- ER_Diagram.png
|
|-- database/
|   |-- schema.sql
|   |-- sample_data.sql
|   |-- basic_queries.sql
|   |-- join_queries.sql
|   |-- views.sql
|   `-- trigger.sql
|
|-- backend/
|   |-- app.py
|   |-- config.py
|   `-- routes/
|
|-- frontend/
|   |-- templates/
|   `-- static/
|
`-- docs/
    |-- Final_Report.pdf
    `-- Presentation.pptx
```

Listing 2: Project Folder Structure

# 6. Weekly Deliverables Summary

| | |
|---|---|
| **Requirement Analysis** | `README.md`, `Requirement_Analysis.pdf` |
| **ER Diagram** | `ER_Diagram.png` |
| **Database Schema** | `schema.sql` |
| **Sample Data** | `sample_data.sql` |
| **Basic SQL Queries** | `basic_queries.sql` |
| **JOIN Queries** | `join_queries.sql` |
| **Views** | `views.sql` |
| **Trigger** | `trigger.sql` |
| **Flask Backend** | `backend/` |
| **Frontend + CRUD** | `frontend/` |
| **Final Report** | `Final_Report.pdf`, `Presentation.pptx` |

# 7. How to Run the Project

```
# Step 1: Clone the repository
git clone https://github.com/your-username/hostel-management-system.git
cd hostel-management-system

# Step 2: Install Python dependencies
pip install flask mysql-connector-python

# Step 3: Setup the database
mysql -u root -p < database/schema.sql
mysql -u root -p < database/sample_data.sql

# Step 4: Configure database credentials in backend/config.py

# Step 5: Run the Flask application
cd backend
python app.py
```

Listing 3: Setup and Run Instructions

Open your browser and visit: `http://127.0.0.1:5000`

# Notes for the Developer

- **Push to GitHub every week** — even partial progress is better than missing commits.
- **Write comments in your SQL files** to explain what each query does.
- **Test the trigger** by trying to insert more students than a room's capacity allows.
- **Use Bootstrap components** (cards, tables, navbar) to make the UI look professional with minimal CSS.
- **Start Flask with a simple / route** before adding more pages — confirm the server runs first.
- **Keep `config.py` separate** from `app.py` for clean database credential management.